

Data Structures - Using Arrays

Based on Goodrich, Tamassian, Mount (2e) 2012

Marcus Birkenkrahe

July 18, 2025

Contents

1	Objectives	2
2	Storing Game Entries in an Array	2
3	PRACTICE Scoreboard Model 1	3
4	PRACTICE Scoreboard Model 2	4
5	PRACTICE Scoreboard Model 3	5
6	Representing high score entries	6
7	Code implementation: High Score Entry	6
8	Using Org-mode to make C++ programming interactive	7
9	PRACTICE Implementation of GameEntry	7
10	PRACTICE Testing a Class Implementation	9
11	Sorting an Array	9
12	Two-dimensional Arrays and Positional Games	9
13	Glossary	9

1 Objectives

- ☐ Examples and functions that use one- or two-dimensional arrays
- ☐ Concept of resizable arrays for C++ **vector** and **string** classes
- ☐ Support for iterators and memory management for array classes

Topics:

- ☐ Storing Game Entries in an Array
- ☐ Sorting an Array
- ☐ Two-dimensional Arrays and Positional Games

2 Storing Game Entries in an Array

- We want to store a sequence of high score entries for a video game in an array. Records for hospital patients or the names of players on a football team would be analogous examples.
- We develop a **Scoreboard** class to store a certain number of high scores (depending on the game).
- Once the score limit is reached, a new score qualifies for inclusion if it is higher than the lowest score on board. Entries will be stored from highest to lowest score. In the case of a tie, the score entered first will appear ahead of another.
- An entry contains a string with the name of the player, and an integer with the player's score. A player may have more than one entry on the high-score board (if they play the game often enough).
- A typical state of a scoreboard with capacity 10, storing 6 current entries:

Mike 1105	Rob 750	Paul 720	Anna 660	Rose 590	Jack 510				
0	1	2	3	4	5	6	7	8	9

3 PRACTICE Scoreboard Model 1

- 1) A scoreboard has capacity five, and the following entries:

Player	Max	Elle	Billy	Robin	
Score	480	465	380	300	

What is the scoreboard's content after Mike plays a game with score 350?

☐

Player	Max	Elle	Billy	Robin	Mike
Score	480	465	380	300	350

☐

Player	Max	Elle	Billy	Mike	Robin
Score	480	465	380	350	300

☐

Player	Max	Elle	Billy	Mike	
Score	480	465	380	350	

Answer:

☒

Player	Max	Elle	Billy	Mike	Robin
Score	480	465	380	350	300

Why? - Mike's score is higher than Robin's so Mike's is added with the fourth highest score, and Robin's becomes the fifth highest.

4 PRACTICE Scoreboard Model 2

- 2) A scoreboard has capacity five, and the following entries:

Player	Billy	Will	Nancy	Dustin	Steve
Score	550	425	350	270	200

What is the scoreboard's content after Chrissy plays a game with score 425?

- ☐

Player	Billy	Will	Chrissy	Nancy	Dustin
Score	550	425	425	350	270
- ☐

Player	Billy	Chrissy	Will	Nancy	Dustin
Score	550	425	425	350	270
- ☐

Player	Billy	Chrissy	Nancy	Dustin	Steve
Score	550	425	350	270	200

Answer:

☒

Player	Billy	Will	Chrissy	Nancy	Dustin
Score	550	425	425	350	270

Why? - Chrissy's score ties Will's for 2nd highest. The rule for equal score is that the older score appears first on the scoreboard, hence Will before Chrissy.

5 PRACTICE Scoreboard Model 3

3) A scoreboard has capacity five, and the following entries:

Player	Lucas	Joyce	Murray	Erica	Hopper
Score	800	650	400	360	240

What is the scoreboard's content after Murray plays a game with score 700?

- ☐

Player	Lucas	Joyce	Murray	Erica	Hopper
Score	800	650	700	360	240
- ☐

Player	Lucas	Murray	Joyce	Erica	Hopper
Score	800	700	650	360	240
- ☐

Player	Lucas	Murray	Joyce	Murray	Erica
Score	800	700	650	400	360

Answer:



Player	Lucas	Murray	Joyce	Murray	Erica
Score	800	700	650	400	360

Why? - Murray's new score is added to the scoreboard in 2nd place, and Murray's original score remains as well.

6 Representing high score entries

- We have to store a **string name** and an **int score** for each entry. We could maintain two (parallel) separate arrays for names and scores.
- What's the problem with this two-array approach?

If we fail to keep the array items aligned then this representation would become inconsistent.

- Instead, we develop a single composite data type that encapsulates both a **name** and a **score**, and a single array of those composite objects.
- What's another advantage of this approach?

If we want to add another field for statistical purposes, e.g. the date, or additional player data, we can adapt our class easily.

7 Code implementation: High Score Entry

- What does this **class** do, and what's in it?

```
class GameEntry {

private:
    string name;
    int score;

public:
    GameEntry(const string& n="", int s=0);
    string getName() const;
```

```
    int getScore() const;
};
```

- Answer:

The code fragment shows a **class** called **GameEntry** that stores **private** information about the **name** and the **score** of a player. It "does" nothing as such.

The **class** has one **constructor** with two default arguments: An empty constant **name** and a zero **score**. There are two **accessor** (aka **getter**) member functions: **getName** extracts the **name**, and **getScore** extracts the score. Both are declared **const** (they're not changing anything).

8 Using Org-mode to make C++ programming interactive

- A few questions on the Org-mode header for the code block fragment:
 1. Can you explain the header?
 2. Is it necessary?
 3. How could you validate your answer?
 1. The header sets up a C++ code block. It wraps the class definition into a **main** program, adds **#include** for **<iostream>** (I/O) and **<string>** header files and **using namespace std;**. Output is suppressed.
 2. This header is not necessary but optional to quickly check that the syntax is correct. **:main**, **:includes** and **:namespaces** arguments could all be put into the code block.
 3. Add **:tangle GameEntry.h** and open (C-x C-f) the file **GameEntry.h** to check.

9 PRACTICE Implementation of GameEntry

- What do we need to bring this **class** to life?

1. Implementations for the three member functions. We can add the implementation to the interface, or factor them out.
 2. A **main** program to test the creation of a **GameEntry** object (with the constructor) and the two getter functions.
- Write down the implementation of the constructor and the two accessor functions now!
 - Solution:

```
GameEntry::GameEntry(const string& n, int s)
: name(n), score(s) { }

string GameEntry::getName() const { return name; }

int GameEntry::getScore() const {return score; }
```

- Explanations:
 1. Externally defined member functions need to be brought in scope with the scope resolution operator (`::`) so that the compiler knows where to look for the **class** declaration.
 2. The constructor initializes **name** and **score** with a list. An alternative implementation would be:


```
GameEntry::GameEntry(const string& n, int s)
{
    name = n;
    score = s;
}
```
 3. The **string** argument is passed as a **reference**. References are alternative names for an (existing) object (see code example).
 4. The code block

Code example:

```
string author = "Samuel Clemens";
string& penName = author;
penName = "Mark Twain";
```



```
cout << author << endl;  
cout << penName;
```

```
Mark Twain  
Mark Twain
```

10 PRACTICE Testing a Class Implementation

- Write down a file to test this!
- How can we get to the test file to run?

11 Sorting an Array

12 Two-dimensional Arrays and Positional Games

13 Glossary

TERM	EXPLANATION
class	A data container
public	Data accessible from anywhere
private	Data accessible only with class member functions
member function	A function inside the class to manipulate data
constructor	A member function to create a class object
accessor/getter	A member function to extract private data